CONFIDENTIAL

UTM UNIVERSITI TEKNOLOGI MALAYSIA | Faculty of Computing

MID TERM TEST II (PRACTICAL)
SEMESTER II, 2024/2025

SUBJECT CODE	:	**SECJ1023**
SUBJECT NAME	:	**PROGRAMMING TECHNIQUE II**
SECTION	:	**1 - 7**
TIME / DURATION	:	**8PM – 10PM / 2 HOURS**
DATE / DAY	:	**14 MAY 2025 / WEDNESDAY**
VENUE	:	**N28, MPK1-4 & MPK8-10**

INSTRUCTIONS

- This is a CLOSED-BOOK practical test. You will attempt the test by coding on a computer.
- You will be provided with some reference resources. You are only allowed to refer to the provided resources.
- References to other resources including any source code, any websites, etc., are strictly prohibited.
- The use of any kind of artificial intelligence tool is strictly prohibited. These tools include ChatGPT, AI-based VS Code extensions such as Co-Pilot, etc.
- You should code offline using a C++ IDE such as VS Code, Dev C++, etc.
- You must use a PC provided in the lab. The use of your own computer is not allowed.
- You will only use e-learning to download the provided resources and submit your answers. Please refer to the e-learning for your section.
- All COMMENTS in your program WILL NOT BE EVALUATED.
- Programs that CANNOT COMPILE will be penalized by 50%.
- Be warned that your program will be subject to a screening process for PLAGIARISM DETECTION.
- NO TECHNICAL ASSISTANCE will be provided during the test. This also applies to problems such as the IDE not working, the program not compiling, etc. If you encounter a problem, you must solve it yourself.
- All questions related to the test question will not be entertained at all. Please read the question carefully.

DOWNLOAD THE <u>RESOURCES PROVIDED</u> FROM ELEARNING

This question paper consists of FIVE (5) printed pages excluding this page.

There is **no starter code** provided for this test. You have to write the program from scratch. Write the program using inline style and use only a single source code file. Indicate the question number (or task number) by writing it as a comment in your code.

Question **[100 Marks]**

Write a complete C++ program that defines and uses a class named **`DroneTask`** to manage individual task data. Declare the class according to the following code snippet:

```
class DroneTask {
  private:
     string taskID;        // a unique identifier for the task
     int durationSec;      // duration in seconds
     double accuracyPct;   // accuracy in percentage (0.0 to 100.0)
     bool critical;        // true if the task is critical
  public:
     // other members should go here
};
```

Complete the code for the class in your program by completing the following tasks:

1. Define multiple constructors as follows:

   a. Define a default constructor that sets the `taskID` to **`"UNKNOWN"`**, `durationSec` to **`0`**, `accuracyPct` to **`0.0`** and `critical` to **`false`**. (4 marks)

   b. Define a constructor that accepts all four attribute values and assigns them to the corresponding data members. (5 marks)

   c. Define a constructor that accepts only the `taskID` and `accuracyPct` values and sets the remaining attributes (`durationSec` and `critical`) to default values (**`0`** and **`false`** respectively). (5 marks)

   d. Define a constructor that accepts a single string in the following format:

      `"taskID|accuracyPct|durationSec|critical"`

      You must extract each value from the string and assign them to the corresponding attributes by using **ONLY** the methods provided by the **C++ string class**. For example, if the string passed is **`"T-AX204|93.3|120|true"`**, then the attributes will be set as

`taskID = "T-AX204"`, `accuracyPct = 93.3`, `durationSec = 120` and `critical = true` (set to **`true`** if the string is **`"true"`** or **`"1"`**). (14 marks)

2. Define a destructor that displays the following message when a `DroneTask` object is destroyed:

   `Task [`**`taskID`**`] is now removed from memory` (2 marks)

3. Define a regular function:

   a. Define a function named **`showTaskSummary()`** that accepts a `DroneTask` object by constant reference and prints the task's details in the following format:

      `Task` **`taskID: durationSec`** `s,` **`accuracyPct`**`% accuracy,` **`CRITICAL/ NON-CRITICAL`**

      Example: `Task` **`T-AX204: 120`**`s,` **`93.3`**`% accuracy,` **`CRITICAL`** (7 marks)

   b. Declare this function as a **`friend`** of the `DroneTask` class. (1 mark)

4. Define the following overloaded operator functions:

   a. Define a greater than operator function to compare two `DroneTask` objects based on their accuracy percentage. It should return `true` if the calling object has a higher accuracy percentage than the object being compared. (3 marks)

   b. Define an equality operator function to compare the `taskID` attributes of two `DroneTask` objects. The comparison must be case-insensitive, meaning that uppercase and lowercase letters should be treated as equal. For example, **`"T-AX101"`** and **`"t-ax101"`** should be considered equal and the function should return `true`. (10 marks)

   c. Define an addition operator that allows an integer value to be added to a `DroneTask` object. This function should add the integer value (the given number of seconds) to the `durationSec` of the task and return the updated object. (4 marks)

   d. Define a **`friend`** function for the output operator to enable printing of `DroneTask` objects using **`cout`**. The output should follow this format:

      `[`**`taskID`**`]` **`durationSec`** `s,` **`accuracyPct`**`% accuracy,` **`CRITICAL/ NON-CRITICAL`**

      Example: `[`**`T-AX204`**`]` **`120`**`s,` **`93.3`**`% accuracy,` **`CRITICAL`**

Use appropriate formatting techniques to ensure the accuracy is displayed with one decimal place. (10 marks)

5. Define several conversion operator functions as follows:

   a. Define a conversion function that allows a `DroneTask` object to be converted to an integer value. This function should return the value of the `durationSec` attribute. (2 marks)

   b. Define a conversion function that allows a `DroneTask` object to be converted to a `double` value. This function should return the `accuracyPct` as a decimal value between 0.0 and 1.0. (2 marks)

   c. Define a conversion function that allows a `DroneTask` object to be converted to a `string` object. The returned string must follow this format:

      `taskID` `(Accuracy:` **`accuracyPct`**`%) -` **`durationSec`** `s`

      Example: **`T-AX204`** `(Accuracy:` **`93.3`**`%) -` **`120`**`s` (5 marks)

   d. Define a conversion function that allows a `DroneTask` object to be evaluated in a `boolean` context. The function should return `true` only if the task is marked as critical and the `accuracyPct` is greater than or equal to 90.0. (3 marks)

6. In the main function of the program, use the `DroneTask` class to perform the following tasks:

   a. Declare an array of four `DroneTask` objects. Initialize each object using a different constructor as defined in Task 1:

      - First element: use the default constructor defined in 1(a).
      - Second element: use the constructor with all parameters defined in 1(b).
      - Third element: use the constructor with only `taskID` and `accuracyPct` as defined in 1(c).
      - Fourth element: use the constructor that accepts a single formatted string as defined in 1(d).

      Refer to the sample program output shown in Figure 1. (5 marks)

   b. Display the details of all four task objects using both the `showTaskSummary()` function and the overloaded output operator. Refer to the example output in Figure 1. (3 marks)

c. Add 20 seconds to the `durationSec` of all task objects using the overloaded addition operator. Then, display the updated task details. Refer to the output format in Figure 1.

(3 marks)

d. Use the overloaded greater than operator to compare all four task objects and identify the one with the highest accuracy percentage. Display the most accurate task using the output operator. Refer to the example in Figure 1. (4 marks)

e. For each task object, test the conversion operator functions defined in Task 5 and print the results. Refer to Figure 1 for the expected output format. (6 marks)

f. Use the equality operator to compare the `taskID` values of two `DroneTask` objects. The comparison should be case-insensitive. Refer to Figure 1 for the expected output.

(2 marks)

```
Task UNKNOWN: 0s, 0% accuracy, NON-CRITICAL
[UNKNOWN] 0s, 0.0% accuracy, NON-CRITICAL
Task T-AX101: 150s, 88.5% accuracy, CRITICAL
[T-AX101] 150s, 88.5% accuracy, CRITICAL
Task T-AX202: 0s, 95.2% accuracy, NON-CRITICAL
[T-AX202] 0s, 95.2% accuracy, NON-CRITICAL
Task t-bx300: 130s, 93.3% accuracy, CRITICAL
[t-bx300] 130s, 93.3% accuracy, CRITICAL

--- Adding 20 seconds to all tasks ---
Task [UNKNOWN] is now removed from memory.
[UNKNOWN] 20s, 0.0% accuracy, NON-CRITICAL
Task [T-AX101] is now removed from memory.
[T-AX101] 170s, 88.5% accuracy, CRITICAL
Task [T-AX202] is now removed from memory.
[T-AX202] 20s, 95.2% accuracy, NON-CRITICAL
Task [t-bx300] is now removed from memory.
[t-bx300] 150s, 93.3% accuracy, CRITICAL

--- Most Accurate Task ---
[T-AX202] 20s, 95.2% accuracy, NON-CRITICAL

--- Conversion Results ---
Duration (int): 20
Accuracy (double): 0.0
String: UNKNOWN (Accuracy: 0.000000%) - 20s
Is high-priority (bool): No

Duration (int): 170
Accuracy (double): 0.9
String: T-AX101 (Accuracy: 88.500000%) - 170s
Is high-priority (bool): No

Duration (int): 20
```

```
Accuracy (double): 1.0
String: T-AX202 (Accuracy: 95.200000%) - 20s
Is high-priority (bool): No

Duration (int): 150
Accuracy (double): 0.9
String: t-bx300 (Accuracy: 93.300000%) - 150s
Is high-priority (bool): Yes


--- Task ID Comparison ---
Different ID

Task [T-AX202] is now removed from memory.
Task [t-bx300] is now removed from memory.
Task [T-AX202] is now removed from memory.
Task [T-AX101] is now removed from memory.
Task [UNKNOWN] is now removed from memory
```

Figure 1: An example output of the program.

-End of Question--